

RViz: a toolkit for real domain data visualization

Hyeong Ryeol Kam² · Sung-Ho Lee³ · Taejung Park⁴ · Chang-Hun Kim¹

© Springer Science+Business Media New York 2015

Abstract In computational science and computer graphics, there is a strong requirement to represent and visualize information in the real domain, and many visualization data structures and algorithms have been proposed to achieve this aim. Unfortunately, the dataflow model that is often selected to address this issue in visualization systems is not flexible enough to visualize newly invented data structures and algorithms because this scheme can accept only specific data structures. To address this problem, we propose a new visualization tool, RViz, which is independent of the input information data structures. Since there is no requirement for additional efforts to manage the flow networks and the interface to abstracted information is simple in RViz, any scientific information visualization algorithms are easier to implement than the dataflow model. In this paper, we provide case studies in which we have successfully implemented new data structures and related algorithms using RViz, including geometry synthesis, distance field representation, and implicit surface reconstruction. Through these cases, we show how RViz helps users visualize and understand any hidden insights in input information.

Keywords Spatial data visualization · Visualization toolkit · Decorator pattern · Lightweight framework

✉ Chang-Hun Kim
chkim@korea.ac.kr

¹ Department of Computer and Radio Communications Engineering, Korea University, Seoul, Korea

² Department of Visual Information Processing, Korea University, Seoul, Korea

³ Onsquare Co., Seoul, Korea

⁴ Department of Digital Media, Duksung Women's University, Seoul, Korea

1 Introduction

Many scientific research studies and computer graphics applications handle data in the real domain (i.e. $\mathbb{R}^2$ for two dimensions and $\mathbb{R}^3$ for three dimensions). In this regard, the ability to visualize real-domain information effectively and simply is very important. For computer graphics, a number of applications rely on real-domain scalar field or vector field, including collision detection, spatial subdivision tree, implicit surface representation, rendering, geometry synthesis, and fluid dynamics, to name just a few. For the mentioned research studies, the main contributions are often related to proposing novel and more efficient data structures to store and visualize real-domain information. This necessitates an efficient visualization method for those under development.

However, the dataflow models [3, 13–15, 22, 30, 34], which are often applied to conventional visualization systems, can only accept data stored in certain data structures. As a consequence, the limitations related to the specific data structure pose huge obstacles for research projects aiming to develop new data structures and algorithms.

In general, the ability to visualize field structures or intermediate results of incomplete algorithms under development is highly required for various purposes, including debugging, acquiring, and explaining insights behind algorithms, and assessing the quality of final results. For more specific instances, when a researcher wants to develop octree-based data structures for textures [2], distance fields [5, 25], or physical entities for fluid simulations [21], she or he should visualize how data are spatially distributed to check validities and problems in developed algorithms and its implementations. Basic visualization tools such as rotation, translation, zoom, cross-sectioning, ray-casting, color-coded rendering, etc. are necessary for an effective review. For computationally expensive operations, some techniques to improve interac-

tivity, such as multithreading, and progressive visualization approaches that first provide low-resolution images and show high-resolution versions later, are required. Moreover, as in some other studies such as [17], researchers need good visualization tools to compare between results from previous techniques and newly proposed ones to claim improvements or advantages. Unfortunately, it is common for researchers to consume more time in designing and implementing such visualization tools than in implementing their ideas and algorithms. To address this issue, we implemented a visualization toolkit called RViz to visualize general real-domain data

1.1 Contributions

In this paper, based on our experience in developing RViz, we discuss issues for the design and architecture of a real-domain data visualization system that accepts arbitrary data structures and algorithms. Our RViz toolkit is a lightweight system that highly modular by separating interfaces and implementations. As a result, developers can easily implement their own visualization toolkit based on guidelines we present in this paper. Also, we present case studies where we have applied our RViz toolkit to computer graphics research projects in which we have been involved recently [16–18], to illustrate how the RViz toolkit contributes to improving development processes.

2 Design goals

In this section, we present the design goals for RViz. These design goals were naturally raised during our research projects [16–18]; we believe the issues we discussed in Sect. 1 can be addressed by achieving these goals.

2.1 Visualization based on production codes

An ideal visualization system should not require additional scripts only for visualization purposes. For example, a developer should be able to ideally apply any codes that implement distance fields [5, 17, 20] to both visualization and other applications that rely on it (e.g. collision detection [8], geometry modeling [1, 18], etc.) without any modification. If not, developers would have to implement prototype codes that are not part of the main implementations (i.e. “production codes”). In this scenario, the production codes could not be visualized for debugging or performance measurement.

2.2 Data structure-agnostic interface

A visualization system should not confine formats of data structures. This is because there are so many methods and data structures to represent real-domain data and we would have difficulties in adding new data structures to systems if a

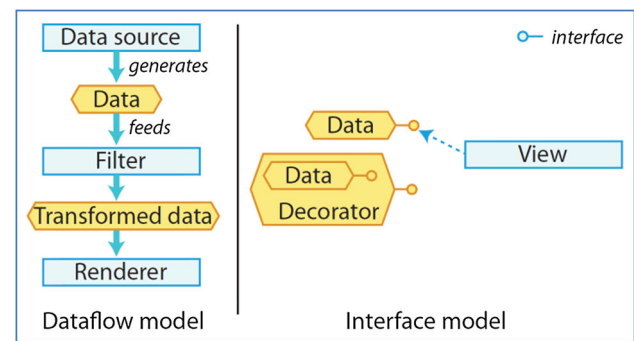


Fig. 1 In the dataflow model (*left*), data are captured in specific data structures and then converted by filters. In contrast, in our interface model (*right*), data is regarded as an implementation of the simple interface. This enables flexibility in choosing data structures and algorithms. The decorator design pattern plays the roles of filters and renderers. In our architecture, there is no need for modules to manage flow networks, since dataflows in our model are handled implicitly within the decorators

toolkit were limited to specific data structures. For instance, there are many methods that can be used to represent implicit surfaces including regular grids [32], binary space partition [38], octree [5], partition of unity [24], and integral moving least squares (MLS) [31]. Of course, the ideal visualization system should be able to handle all of those and others that would be introduced in the future. To satisfy the discussed requirements, our data model was based on an architecture that separates interface and implementation (See Fig. 1).

2.3 Lightweight framework

To implement the ideal visualization system in the same programming language environment as the production codes, it would be beneficial to make a core part as small as possible. This design rule eliminates difficulties in porting a large number of codes in new programming language environments. The fully functioning RViz toolkit codes were implemented in Java but we implemented prototypes easily in other programming languages including C++ and Scala [26] because the core interfaces in our system were very simple. We discuss issues of selecting a programming language for the visualization system in Sect. 6.

2.4 Dimension-agnostic data model

Because two-dimensional information is easier to visualize and calculate than the three-dimensional version, it is common that developers and researchers test their new algorithms with two-dimensional data structures and then proceed with their method to three-dimensional data. Therefore, if we could handle both two- and three-dimensional data with the same codes, the whole research processes would be highly improved.

3 Related work

Many visualization systems [13,14,30,33] for spatial data construct flow networks based on predefined data structures and the renderer at the final stage. In the previous methods, spatial data are often represented by data structures based on regular grids. However, some parts of important information can be lost during the sampling process and much memory is consumed with three-dimensional regular grids. Also, many previous visualization systems provide a large set of libraries. The problem is that most of related classes have complicated hierarchies, and too many methods. Because of this complexities, third-party developers often have trouble in adding necessary features to such heavyweight architectures. Also, traditional heavyweight toolkits handle data sources, filters, and renderers separately. In contrast, in our design approach the simplest model that represents one vector field encompasses all concepts for data sources, filters, and renderers. In other words, our design philosophy is based on the idea that “the simplest explanation is most likely the correct one” [23].

Gamma et al. [6] describe software design patterns as “descriptions of communicating objects and classes that are customized to solve design problems within a particular context.” The decorator design pattern [6] is widely used in the GUI toolkit [36] and for information visualization [11,12], but is not fully studied in representation and management of real-domain scalar and vector field information. In this paper, we show various visualization techniques can be implemented based on the decorator pattern; also, we prove that the decorator pattern helps visualize spatial data more easily.

Although many visualization systems are implemented in C++, we needed special methods to change properties of the necessary components during runtime, because the standard C++ specifications do not provide the “reflection” feature. In contrast, Java provides a reflection natively, so we could implement runtime property injection directly in Java environment. GeoVISTA [33] proposed a geoscientific visualization toolkit that wires objects based on JavaBeans. To implement JavaBeans properties, boilerplate codes are required for getters and setters. To reduce this difficulty, Groovy [7] and the Scala language [28] provide ways for implicitly creating getters and setters.

4 RVizdata model

As illustrated in Fig. 1, RViz allows a View module to access data objects via interfaces. Such interfaces should be as simple as possible and effectively satisfy requirements from specific visualization applications. In this Section, we explain the interfaces in our RViz toolkit.

4.1 Main interface

In our toolkit, the most basic data model is vector field, which is defined with a following Groovy interface.¹

```
public interface VectorField {
    double[] getValues(double... x) }
```

A View module receives the implementations of *VectorField*, calls *getValues(x)* per each pixel, and rasterizes by determining RGB color values to the first three channels. For example, the following code provides the image shown in Fig. 2a:

```
public class SimpleField implements VectorField {
    public double[] getValues(double... x) {
        return x
    }
}
RViz.view(new SimpleField())
```

In this manner, *getValues* contains codes to calculate and return values to visualize. There are no limitations and requirements in the data structures and algorithms used in the implementations. Also, the dimension of the argument *x* in *getValues(x)* is flexibly determined according to the context. Again, this flexibility means that the data object can be implemented to handle either two-dimensional information only or both two- and three-dimensional information simultaneously.

If developers want to represent scalar fields, this function returns an array with only one element. The following code renders distance fields for a sphere:

```
public class SphereDistance implements VectorField {
    double radius = 0.2
    public double[] getValues(double... x) {
        Math.sqrt(x[0]*x[0] + x[1]*x[1] + x[2]*x[2]) - radius
    }
}
```

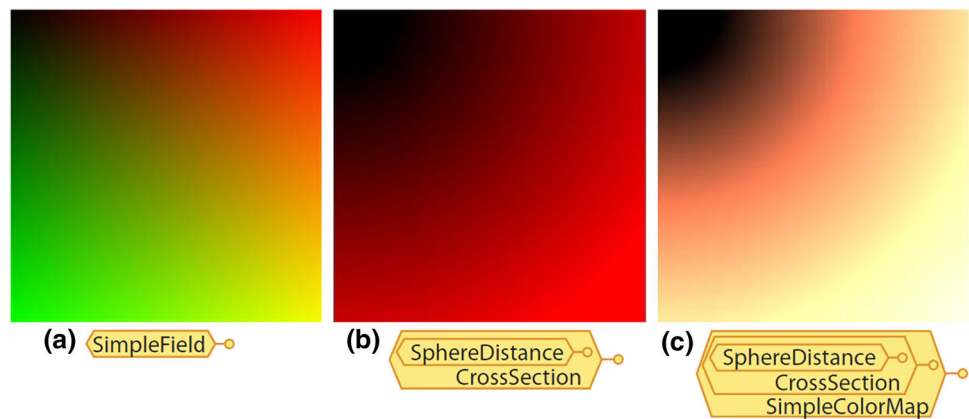
As a JavaBean property, the parameter *radius* can be modified on our toolkit’s GUI in runtime, and the view will be redrawn immediately as the parameter changes.

4.2 Decorators

We can let one implementation of *VectorField* use another data object. The following code shows a decorator that converts three-dimensional fields into two-dimensional cross-sectional views. The result is shown in Fig. 2b.

¹ The Groovy language [8] has similar syntax to Java but provides simpler ways for representations, so we applied Groovy to all code lists if not mentioned otherwise.

Fig. 2 Images rendered based on the interface *VectorField*. These images were visualized by assembling a chain of the decorators



```
public class CrossSection implements VectorField {
    VectorField field
    double z
    public double[] getValues(double... x) {
        return field.getValues(x[0], x[1], z)
    }
}
RViz.view(new CrossSection(field:new SphereDistanceField()))
```

Users can adjust the parameter z using GUI in runtime. The decorators have similar concepts and goals as the filters in the dataflow model [29] in that original fields are converted to other fields but they are different in implementation complexity. If developers want to implement filters in the dataflow model, the algorithms to be implemented in the filters should (1) read values from input data objects, (2) make calculations, (3) store results in output data objects, and (4) all the steps should be repeated for every data element. Unlike this procedure, in our method the decorators simply implement the calculation routine (i.e. step 2 in the dataflow model) making the development simpler.

4.2.1 Example 1: color maps

We can express simple color maps as follows:

```
public class SimpleColorMap implements VectorField {
    VectorField field
    public double[] getValues(double... x) {
        double val = field.getValues(x)[0]
        return [val*3, val*1.5, val]
    }
}
RViz.view(new SimpleColorMap(field:new CrossSection(
    field:new SphereDistanceField())))
```

The image rendered by the above code is shown in Fig. 2c.

4.2.2 Example 2: rendering

Other renders including ray-tracers can be implemented with the decorators. In these implementations, *getValues(x)* shoots and traces rays through a given pixel location x , and then returns three channel color values (See Fig. 3). Since the rendering properties, such as positions for lights and cameras,

belong to the decorator objects, views can be implemented more simply and we can manage rendering algorithms more modular. Based on the discussed advantages, the RViz toolkit can be applied effectively to processes for designing and assessing rendering algorithms. Table 1 lists some of the decorator samples that are used most frequently.

4.3 Retrieving the size of the pixel

To implement algorithms like super-sampling, mipmapping, and segment rendering, we need to let data objects know the size of pixels. To this end, we created an extended vector field interface as follows:

```
public interface VectorFieldEx {
    double[] getValues(double[] x, double[] ext) }
```

Pixel sizes are sent to the function argument *ext*. The only requirement for the View module is that it visualizes *VectorFieldEx*. To render data objects in *VectorField* type, the following simple adaptors can be added and returned to the View module.

```
public class VectorFieldAdaptor extends VectorFieldEx {
    VectorField field
    double[] getValues(double[] x, double[] ext) {
        return field.getValues(x)
    }
}
```

4.3.1 Example 1: supersampling

The supersampling technique [4] reduces aliasing artifacts near surfaces with large differences in color distribution. To create a decorator that supersamples field values, we need to determine the size of the point spread function (PSF). To this purpose, our supersampling decorator receives the size of PSF from the *ext* argument in the *VectorFieldEx* and performs 2×2 uniform sampling. Figure 3 shows the images rendered with our supersampling decorator.

Fig. 3 Scenes created by sphere tracing [10] and supersampling distance fields [17]. As shown here, the supersampling algorithm can be easily added as a decorator

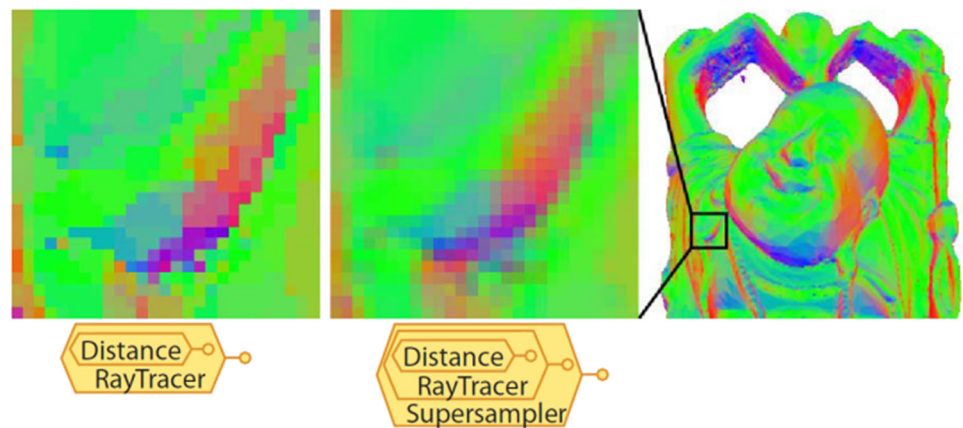
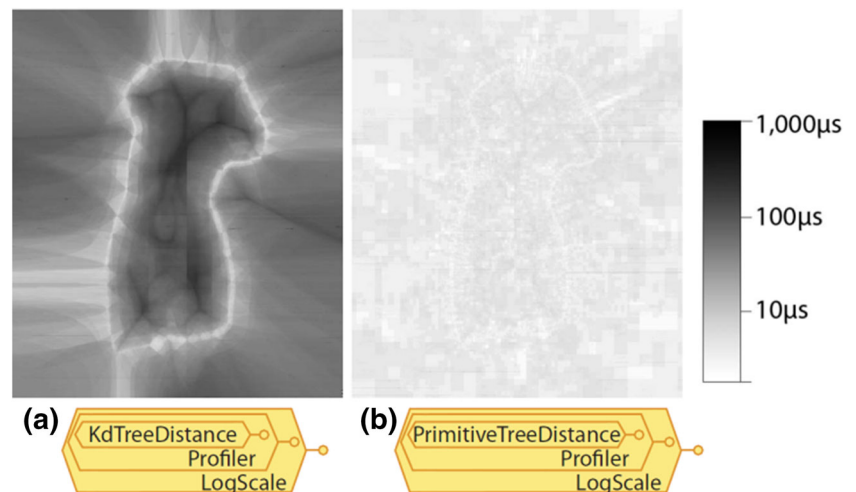


Table 1 Example implementations of *VectorField*

Name	Description
Sphere Distance	A distance field represents a sphere
Mandelbrot	A scalar field represents a Mandelbrot set
LinearGrid	A linearly interpolated image sampled from the regular grid
Bruteforce Triangle Distance	A distance field from a triangle soup
KdTree Triangle Distance	A distance field from a triangle soup accelerated using a <i>kd</i> -tree
PrimitiveTreeTriangleDistance	A distance field from triangle soup accelerated using a primitive tree [21]
PolygotisoupMLS	An implicit surface representation interpolated using moving least squares [20]

Fig. 4 Performance comparison using a *Profiler* decorator. The *Profiler* decorator repeatedly measures times taken to evaluate the field and returns the median. In this example, we can easily see that computational costs for calculating distance fields based on *kd*-tree are higher near concavities



5 Case studies

In this Section, we discuss how RViz can be utilized in real research projects based on our experience in developing new algorithms [16–18].

5.1 Distance field representation

The adaptively sampled distance field (ADF) technique [5] provides a representation method to quickly access signed

distance fields by storing distance values in octreenodes. In our study, we tried to observe and solve errors caused in storing and interpolating sample values in ADF. To improve ADF, we introduced a method for storing geometry primitives, not specific distance values in octree nodes [17].

To compare the performance, we have implemented a *Profiler* decorator that ran the *getValues(x)* operations for specified number of times and returned the median value. Figure 4 compares running time of two different algorithms [17,39] to calculate distance fields. In this figure, we can easily observe not only overall performance discrepancy, but

Fig. 5 Error comparison in distance fields using an *Offset* decorator. The *Offset* decorator adds a constant value to the field. We can clearly see that errors are larger in the regions further from the surfaces in the Lefebvre and Hoppe's method [20] (right)

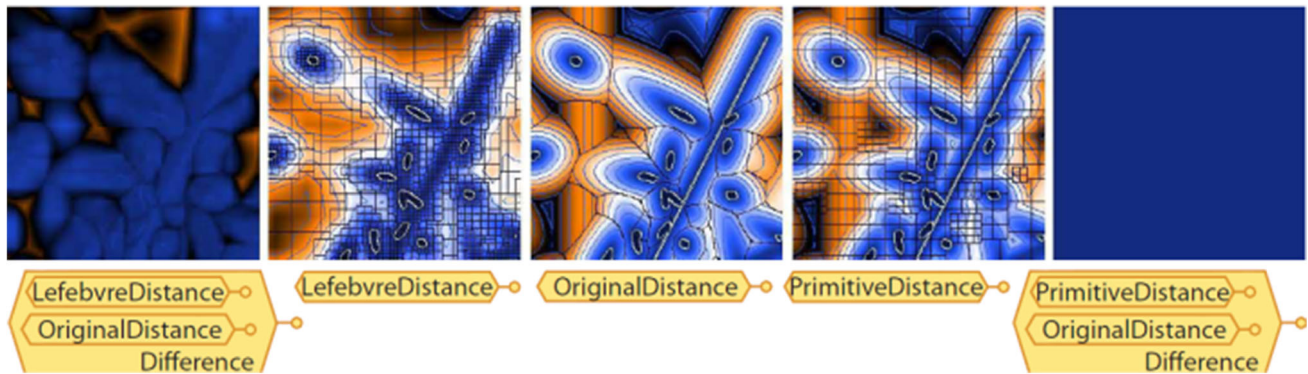
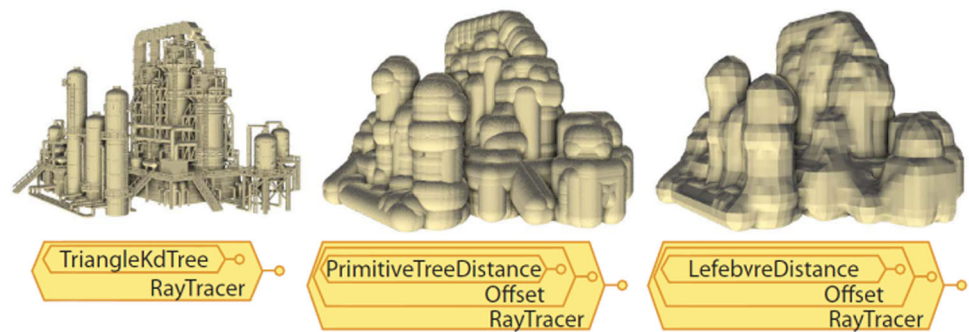


Fig. 6 Discrepancy comparison using a *Difference* decorator. The *Difference* decorator returns discrepancy between two data objects

also local performance degenerations near concavities in the kd-tree-based algorithm (Fig. 4, left).

We have applied a method to render distance field with some offset to distinguish errors visually. In Fig. 5, we have assembled a simple decorator that adds constant offsets for the two algorithms for comparison [17, 20] and renders final images using a decorator implementing the sphere tracing method [10]. We can see that errors are larger in the regions far from the surfaces in the Lefebvre and Hoppe's method [20].

Also, we could easily visualize the discrepancies between any given original data and approximations using the decorator pattern. Figure 6 shows such discrepancies for distance fields using a *Difference* decorator.

5.2 Geometry synthesis

Texture synthesis is the process of algorithmically generating a high-resolution texture from a small input example [37]. We have extended one of the two-dimensional texture synthesis techniques [19] to synthesize three-dimensional distance fields [18]

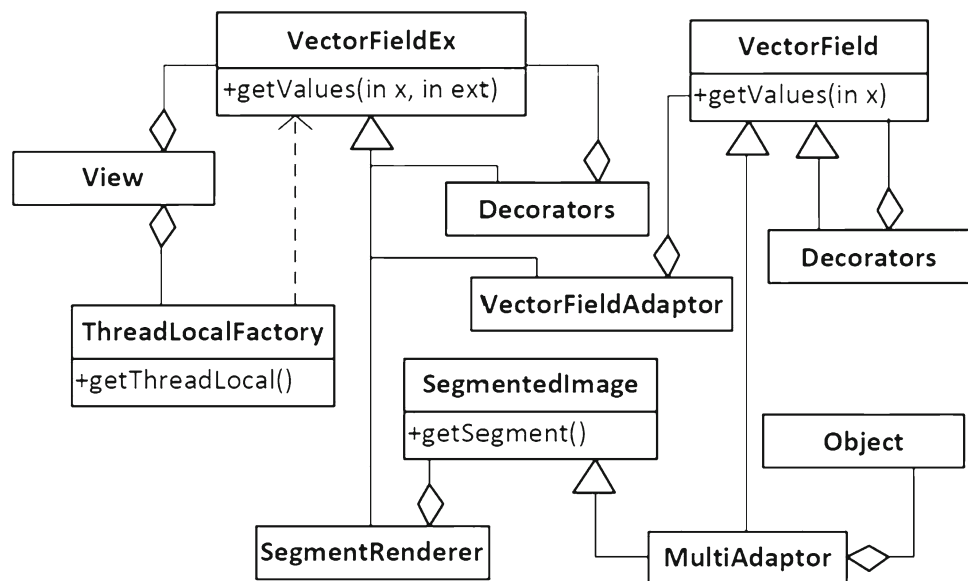
For applications that handle large-scale data such as [18], it is very important to shorten the computational time. To reduce time, we have developed an algorithm and basic data structure to synthesize three-dimensional geometry information near surfaces of distance fields.

5.3 Integral moving least squares for surface representation

Shen et al. [31] proposed a technique to construct implicit surfaces from triangle soups by introducing integrations over triangles in MLS equations. This technique does not provide analytical solutions for the double integrations over triangles. Also, it needs iterative adjustment to encompass the original triangle soup models in approximation mode. Because of such problems, this technique consumes considerable computation time. To address this issue, we have analyzed the original work [31] and found a method to improve the performance [16].

One of us began this study by analyzing the original method, and he needed the ability not only to render our own implicit surfaces, but also to visualize complex continuous fields generated in the original technique. Initially, he tried to implement another proprietary visualization tool for this purpose only, but migrated to the RViztoolkit after realizing it is easier to use. It was quite quick and easy to implement ideas in the RViz framework because of its simple design for data model and versatile GUIs. In our experience, we could avoid any learning challenges that are often involved in expanding features of toolkits with heavy-weighted class hierarchy. We were able to provide a new algorithm to our toolkit by only learning how to use simple *VectorField* interfaces.

Fig. 7 Class diagram for RViz classes. This diagram shows *VectorFieldEx* to represent data objects, *View* to use *VectorFieldEx*, and other optional classes



6 Conclusion and future work

We present RViz, a toolkit aimed for visualizing real-domain data generated by arbitrary data structures and algorithms. The RViz toolkit is lightweight and independent of data structures and algorithms. We have designed the RViz toolkit architecture in such a way that toolkit developers can implement only a small set of features, and each feature is simple and modular, thus this toolkit can be easily ported to various platforms. As a result, we can easily visualize algorithms and data structures, so that it helps developers debug and profile performance of production codes. Figure 7 illustrates the summarized class relationships in the RViz toolkit.

In this paper, we show some examples in which the benefits of RViz toolkit are obvious. We expect our architecture to be widely used for various creative purposes in the future.

One of the limitations in our proposed framework is that the interfaces discussed in Sect. 4.1 do not provide temporal or spatial coherence currently, so that it is impossible to implement algorithms based on it. Therefore, we plan to expand our framework to embrace the advanced rendering techniques including the packet based raytracing [27], raytracing for dynamic scenes [35], and the multipass rendering algorithms such as progressive photon mapping [9]. On the other hand, the auto completion technique [15] would help improve convenience for frequently used parameters when the data object is manipulated with GUIs.

Acknowledgments This research was supported by Basic Science Research Program through the National Research Foundation of Korea(NRF) funded by the Ministry of Education, Science and Technology (No. 2011-0017595). This work was supported by the Technology Innovation Program (Industrial Strategic technology development program, 10035619) funded by the Ministry of Knowledge Economy(MKE, Korea).

References

1. Bærentzen, J. A., & Christensen, N. J. (2001). A technique for volumetric CSG based on morphology. In *Proceedings of the International Workshop on Volume Graphics* (pp.71–79).
2. Benson, D., & Davis, J. (2002). Octree textures. *ACM Transactions on Graphics*, 21, 785–790.
3. Childs, H., Brugger, E., Bonnell, K., Meredith, J., Miller, M. & Whitlock, B., et al. (2005). A contract based system for large data visualization. *IEEE Visualization*, 25.
4. Cook, R. L. (1986). Stochastic sampling in computer graphics. *ACM Transactions on Graphics (TOG)*, 5, 51–72.
5. Frisken, S.F., Perry, R.N., Rockwood, A.P., & Jones, T.R. (2000). Adaptively sampled distance fields: a general representation of shape for computer graphics. *ACM SIGGRAPH* (249–254).
6. Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1995). *Design patterns: Elements of reusable object-oriented software*. Boston: Addison-Wesley Longman Publishing Co., Inc.
7. Groovy community. Groovy programming language. <http://groovy.codehaus.org>
8. Guendelman, E., Bridson, R., & Fedkiw, R. (2003). Nonconvex rigid bodies with stacking. *ACM Transactions on Graphics*, 22, 871–878.
9. Hachisuka, T., Ogaki, S., & Jensen, H. W. (2008). Progressive photon mapping. *ACM Transactions on Graphics*, 27, 130:1–130:8.
10. Hart, J. C. (1996). Sphere tracing: a geometric method for the antialiased ray tracing of implicit surfaces. *The Visual Computer*, 12(10), 527–545.
11. Heer, J., & Agrawala, M. (2006). Software design patterns for information visualization. *IEEE Transaction on Visualization and Computer Graphics*, 12, 853–860.
12. Heer, J., Card, S.K. & Landay, J.A. (2005). Prefuse: a toolkit for interactive information visualization. In *Proceedings of SIGCHI conference on Human factors in computing systems* (pp. 421–430).
13. IBM. OpenDX. <http://www.research.ibm.com/dx>
14. Kitware. Visualization toolkit. <http://www.vtk.org>
15. Koop, D., Scheidegger, C. E., Callahan, S. P., Freire, J., & Silva, C. T. (2008). VisComplete: automating suggestions for visualization pipelines. *IEEE Transaction on Visualization and Computer Graphics*, 14(6), 1691–1698.

16. Park, T., Lee, S.-H., & Kim, C.-H. (2011). Analytic Solutions of Integral Moving Least Squares for Polygon Soups. PrePrint: *IEEE transactions on Visualization and Computer Graphics*.
17. Lee, S. -H., Park, T. & Kim, C. -H. (2011). Primitive Trees for Precomputed Distance Queries. *Technical paper*.
18. Lee, S. -H., Park, T., Kim, J. -H., & Kim, C. -H. (2011). Adaptive synthesis of distance fields. PrePrint: *IEEE transactions on Visualization and Computer Graphics*.
19. Lefebvre, S., & Hoppe, H. (2005). Parallel controllable texture synthesis. *Transactions on Graphics*, 24(3), 777–786.
20. Lefebvre, S., & Hoppe, H. (2007). Compressed random-access trees for spatially coherent data. In *Proceedings Eurographics Symp. on Rendering*.
21. Losasso, F., Gibou, F., & Fedkiw, R. (2004). Simulating water and smoke with an octree data structure. *ACM Transactions on Graphics*, 23, 457–462.
22. MeVis, MeVisLab: A development environment for medical image processing and visualization. <http://www.MeVisLab.de>
23. Occam, William of. Occam's razor. http://en.wikipedia.org/wiki/Occam%27s_razor
24. Ohtake, Y., Belyaev, A., Alexa, M., Turk, G., & Seidel, H.-P. (2003). Multi-level partition of unity implicits. *ACM SIGGRAPH*, 22(3), 463–470.
25. Park, T., Lee, S. -H., Kim, J. -H. & Kim, C. -H. (2010). Cuda-based signed distance field calculation for adaptive grids. In *Proceedings of the IEEE International Conference on Computer and Information Technology* (pp.1202–1206).
26. Pollak, D. (2009). *Beginning Scala*. Dordrecht: Springer.
27. Reshetov, A., Soupikov, A., & Hurley, J. (2005). *Multi-level ray tracing algorithm*. *ACM SIGGRAPH*, 24, 1176–1185.
28. Scala development team. Scala programming language. <http://scala-lang.org>
29. Schroeder, W.J., Martin, K.M. & Lorensen W.E. (1996). The design and implementation of an object-oriented toolkit for 3d graphics and visualization. *IEEE Visualization*, PP. 93–100.
30. Scientific Computing and Imaging Institute (SCI). SCIRun: A scientific computing problem solving environment. <http://www.scirun.org>
31. Shen, C., O'Brien, J.F. & Shewchuk, J.R. (2004) Interpolating and approximating implicit surfaces from polygon soup. In *ACM SIGGRAPH* (pp.896–904).
32. Sud, A., Govindaraju, N.K., Gayle, R. & Manocha D. (2006). Interactive 3d distance field computation using linear factorization. In *Proceedings of the ACM symposium on Interactive 3D Graph. and Games (SI3D)* (pp. 117–124).
33. Takatsuka, M., & Gahegan, M. (2002). GeoVISTA Studio: A codeless visual programming environment for geoscientific data analysis and visualization. *Computational Geoscience*, 28, 1131–1144.
34. Upson, C., Faulhaber, Jr., T., Kamins, D., Laidlaw, D.H., Schlegel, D., Vroom, J., Gurwitz, R. & van Dam A. *The application visualization*.
35. Wald, I., Günther, J. & Slusallek P. (2004). Balancing considered harmful - faster photon mapping using the voxel volume heuristic. *Computer Graph. Forum*, 22(3).
36. Wald, I., Mark, W.R., Günther, J., Boulos, S., Ize, T., Hunt, W., et al. (2009). State of the art in ray tracing animated scenes. *Computer Graphics Forum*, 28(6), 1691–1722.
37. Walrath, K., Campione, M., Huml, A., & Zakhour, S. (2004). *The JFC swing tutorial: A guide to constructing GUIs*. Redwood City: AddisonWesley Longman Publishing Co., Inc..
38. Wei, L.-Y., Lefebvre, S., Kwatra, V. & Turk G. (2009). State of the art in example-based texture synthesis. In *Eurographics 2009, State of the Art Report*.
39. Wu J., Kobbelt L. (2003). Piecewise linear approximation of signed distance fields. In *Proceedings of the Vision, Modeling, and Visualization* (pp.513–520).



Hyeong Ryeol Kam received B.A. and M.S. in Computer Science from Korea University in 2008 and 2010, respectively. And he is a Ph.D. candidate in Computer Science in Korea University. He is interested in neuroimaging analysis and applications of geometry.



Sung-Ho Lee received B.A., M.S. and Ph.D. degrees in Computer Science from Korea University, in 2005, 2007 and 2011 respectively. He is currently running a startup providing mobile services for people playing music. <https://www.linkedin.com/in/sungho>.



Taejung Park has worked as a research professor in the BK21 program at Korea University since 2006. He received B.A. and M.S. degrees in Electrical Engineering from Seoul National University in 1997 and 1999, respectively. He designed a data structure for a direct matrix solver for FEM simulation for semiconductor devices. After working with two small start-up technology businesses in Korea in 1999 and 2002, he received a Ph.D. from Seoul

National University for 3D mesh compression in 2006. His current research interests include parallel numerical simulation techniques and nonlinear interpolation methods from the viewpoint of information technology.



Chang-Hun Kim is a Professor in the Department of Computer Science and Engineering at Korea University. He received his B.A. degree in Economics from Korea University in 1979. After graduation, he joined the Korea Advanced Institute of Science and Technology (KAIST) as a Research Scientist, where he was involved in many national research projects in the area of Computer Aided Design and Geometric Modeling. He received his Ph.D. from

the Department of Electronics and Information Science, Tsukuba University, Japan, in 1993. During 1993–1995, he headed the Human Interface and Graphics Laboratory for the System Engineering Research Institute (SERI). His current research interests include fluid animation and mesh processing. He is a member of IEEE Computer Society and ACM.